

Castillo en el aire – Despliegue de aplicación web en nube por una mejor salud mental

Autor: César García Periañez

2º CFGS Administración de Sistemas Informáticos en la Red
Proyecto Intermodular

IES Albarregas

Fecha de entrega: 01-06-2026



Agradecimientos

A aquéllos que me apoyaron cuando quise dedicarme a lo que realmente me gusta, aún a riesgo de echar años de carrera por la borda. Sabéis quiénes sois. Os quiero

A los profesores del IES Albarregas que me han guiado durante estos dos años de magnífico aprendizaje profesional y personal, especialmente a aquellos que fueron más allá de la mera presencia en el aula.

A mi sobrina, unas disculpas, por no poder pasar más mañanas en el parque con ella.

"Que nadie te diga que no se puede."

Índice

Agradecimientos	1
Introducción	4
Vista general del proyecto	6
Capturas demostrativas	7
Stack de la aplicación web	12
Frontend: Nginx sirviendo un build de ReactJS	12
Backend: Django a través de Gunicorn	13
Base de datos: MariaDB	13
Caché y almacén de sesiones: Redis	13
Almacenamiento persistente de imágenes: AWS S3	14
Diseño de la base de datos	15
Django y el ORM	15
Los modelos de datos	15
Modelo ER (Link a Excalidraw)	16
Diagrama de relaciones	17
Ejemplo: flujo completo de una reserva en la base de datos	18
El panel de administración de Django	20
Utilidades de base de datos	21
Arquitectura AWS	22
Virtual Private Cloud (VPC)	22
Subredes	23
Flujo de tráfico	24
Escalabilidad	24
Security Groups y Network ACLs	25
Orquestación y scripts de despliegue	26
Configuración del entorno	26
Infraestructura local (Windows Server)	30
Windows Server - Active Directory	30
Estructura organizativa - Grupos globales	30
Estructura organizativa - Grupos dentro de unidades organizativas anidadas:	31
Windows Server - Otros servicios integrados	32
Servicios en Windows Server 2022	32
Configuración de Active Directory	33
Seguridad	34
Descripción general	34
Análisis de riesgos	34
Seguridad de red por capas	34
Protección de datos	35

Desarrollo de software-----	37
Django (Python)-----	37
ReactJS-----	39
i18n (Internacionalización)-----	41
Validación en el lado del cliente-----	42
GIF de precarga-----	43
Contraseñas con hash en el registro-----	43
Scripting en Bash-----	44
Posibles mejoras futuras-----	45
Seguridad y cumplimiento normativo-----	45
Integración de pagos (Stripe)-----	45
Integración de videollamadas (Google Meet API)-----	46
OAuth (inicio de sesión social)-----	46
Balanceo de carga nativo de AWS (ALB)-----	46
Orquestación mediante GitHub Actions o Ansible-----	46
Gestión de DNS (Route 53)-----	47
Aplicación de monitorización de pacientes-----	47
Conclusión general-----	48
Bibliografía-----	49
Anexo I: Requisitos mínimos y cómo se cumplen-----	50
Administración de sistemas operativos-----	50
Python-----	51
Seguridad y alta disponibilidad-----	51
Diseño, gestión y administración de la base de datos-----	52
Implementación de aplicaciones web-----	53
Servicios en red-----	54
Anexo II: Imágenes ampliadas-----	55

Proyecto Intermodular – ASIR2

Introducción

Solo en España, casi 5.000 personas ingresan al año en tratamiento de adicciones de conducta (apuestas, redes sociales, videojuegos...). ‘GetBetter’ es una plataforma web SaaS (software as a service) diseñada para clínicas, psicólogos y pacientes. Conecta a psicólogos con sus pacientes en un entorno digital profesional y estructurado.

La visión a largo plazo del producto es especializarse en pacientes con **adicciones conductuales**, un campo que las plataformas médicas genéricas no atienden adecuadamente. La arquitectura y el modelo de datos han sido diseñados para ser extensibles en esa dirección, especialmente en lo relativo al seguimiento de pacientes, el historial de sesiones y la posibilidad de integraciones con terceros como videollamadas y pasarelas de pago.

En estos momentos, la plataforma permite a los psicólogos gestionar su disponibilidad publicando franjas horarias de cita, y permite a los pacientes consultar esas franjas y solicitar reservas. Los psicólogos pueden confirmar, rechazar o cancelar citas, y mantener notas clínicas privadas por sesión junto con notas visibles para el paciente. Cada perfil de psicólogo incluye un número de colegiado profesional que debe ser verificado por un administrador antes de que el psicólogo pueda comenzar a gestionar citas.

Además de la plataforma, se simula la estructura interna de la empresa ficticia administradora de GetBetter mediante herramientas de virtualización.

El proyecto se enmarca como proyecto intermodular final para el ciclo formativo de grado superior de ASIR (Administración de Sistemas Informáticos y Redes) del instituto IES Albarregas.

En la presente documentación, se hace hincapié en los aspectos técnicos informáticos del funcionamiento de la empresa.

Vista general del proyecto

La plataforma web se despliega utilizando las siguientes tecnologías:

- Debian-Nginx - Balanceador de carga y proxy inverso
- Nodos 'App' (Cada uno levanta un Docker Compose)
 - Debian-Nginx, sirve ReactJS y Tailwind
 - En el proceso de creación de la imagen Docker, se compila ReactJS
 - Imagen Python-Slim con Django y Gunicorn
- Debian-MariaDB
 - Almacenamiento de datos persistente
- Debian-Redis
 - Almacenamiento de memoria en caché para persistencia de sesiones durante balanceo de carga)
- AWS S3 Bucket
 - Almacenamiento de imágenes compartido
- Debian adicional en red pública
 - Utilizado como 'jump-host' o bastión para acceder al resto de hosts

Adicionalmente, existe un equipo Windows Server 2022 con los siguientes servicios en red virtualizados:

- DHCP
- DNS
- AD (Con usuarios, GPOs, OUs, grupos...)
- FTP

Capturas demostrativas

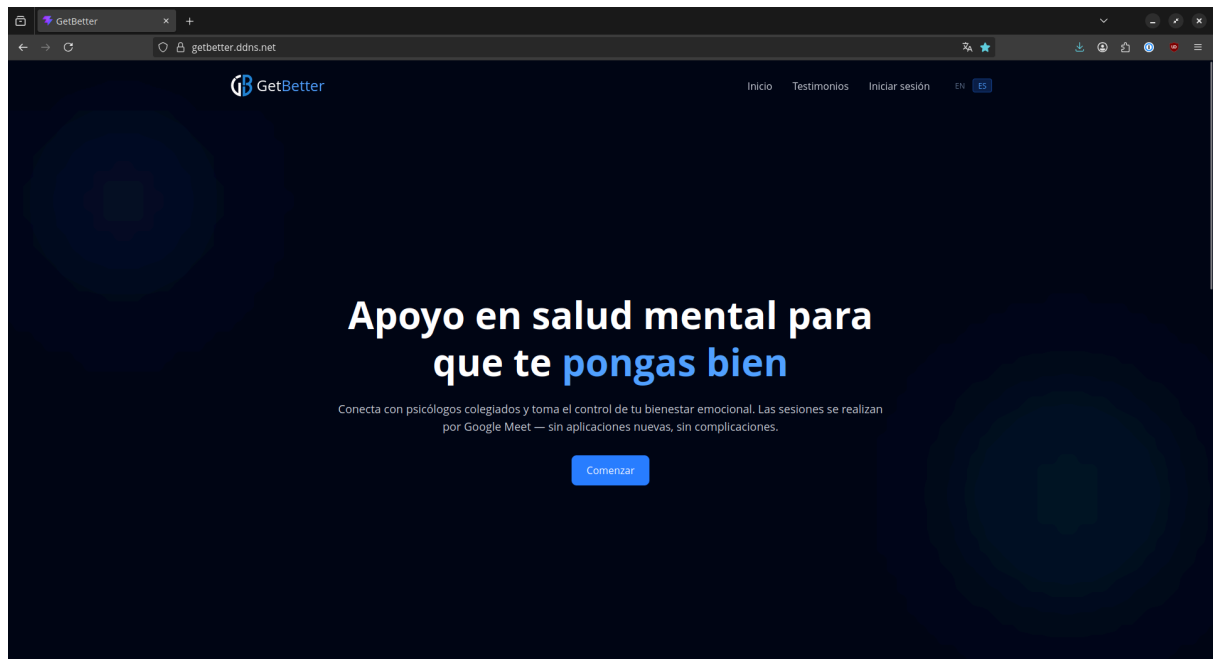


Imagen 1. Captura de pantalla de app en producción (landing page)

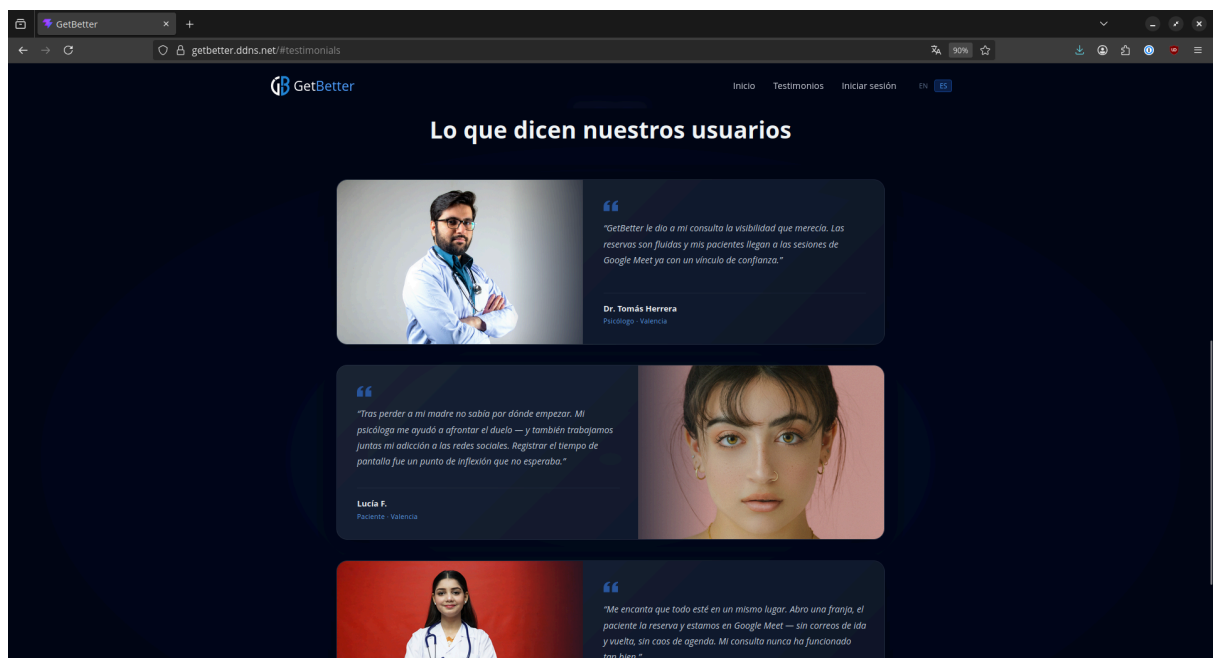


Imagen 2. Captura de pantalla de app en producción (landing page)

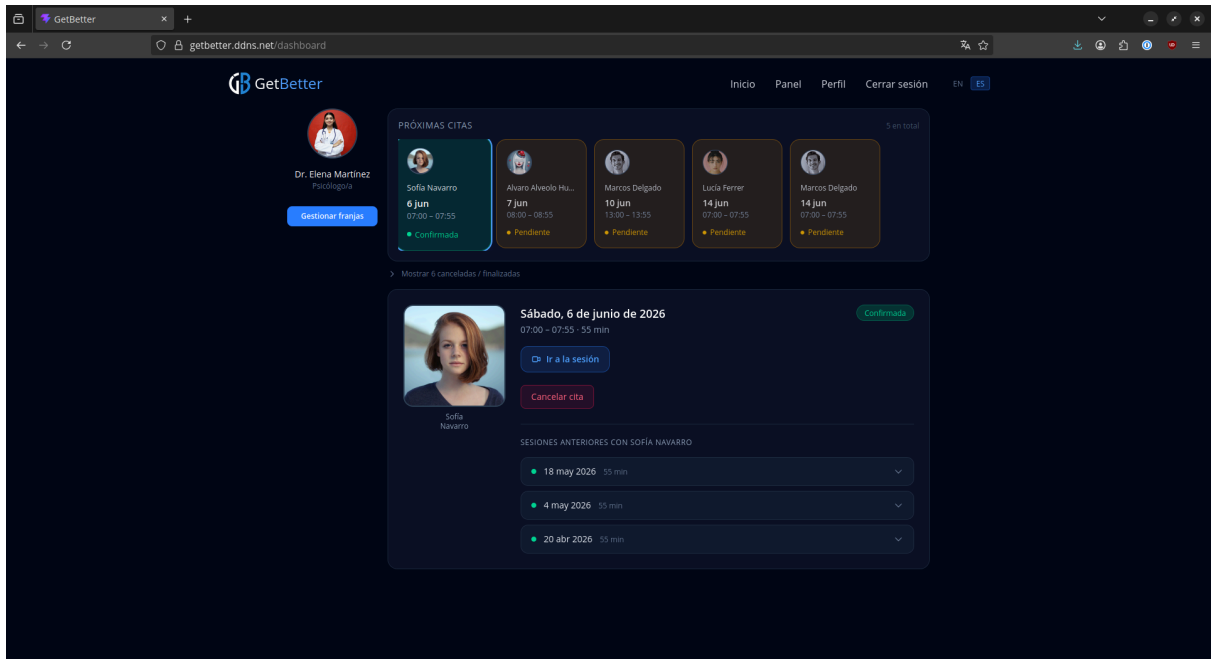


Imagen 3. Captura de pantalla de app en producción (panel de sesiones)

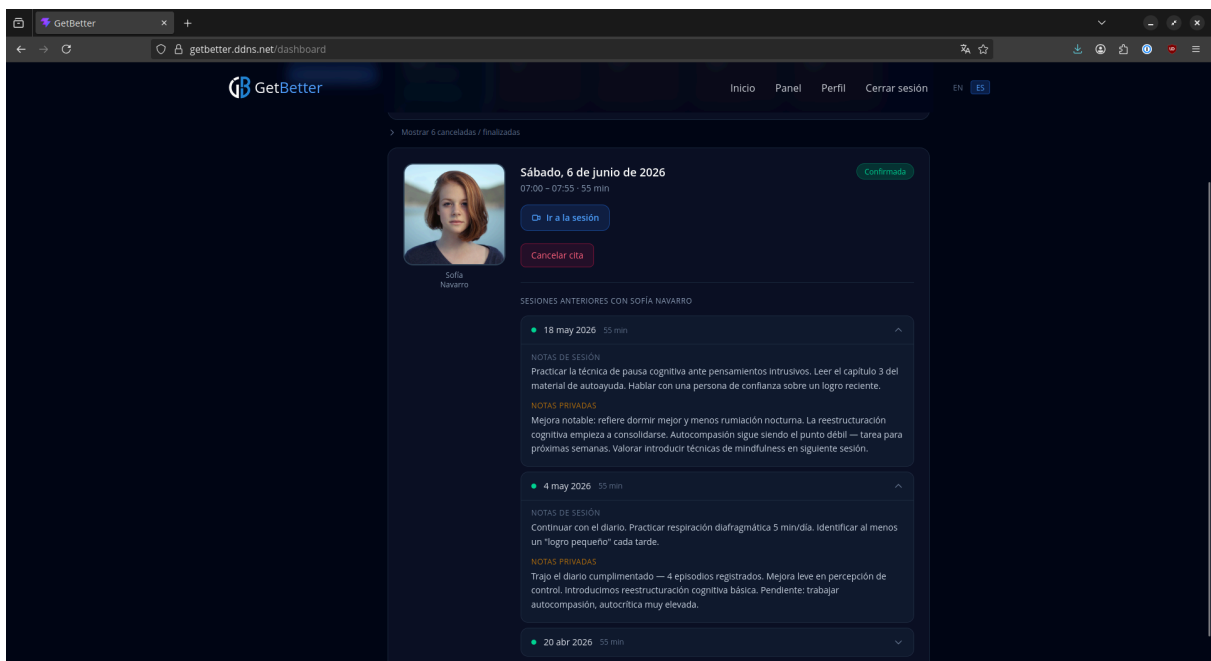


Imagen 4. Captura de pantalla de app en producción (sesión en detalle y apuntes de sesiones pasadas)

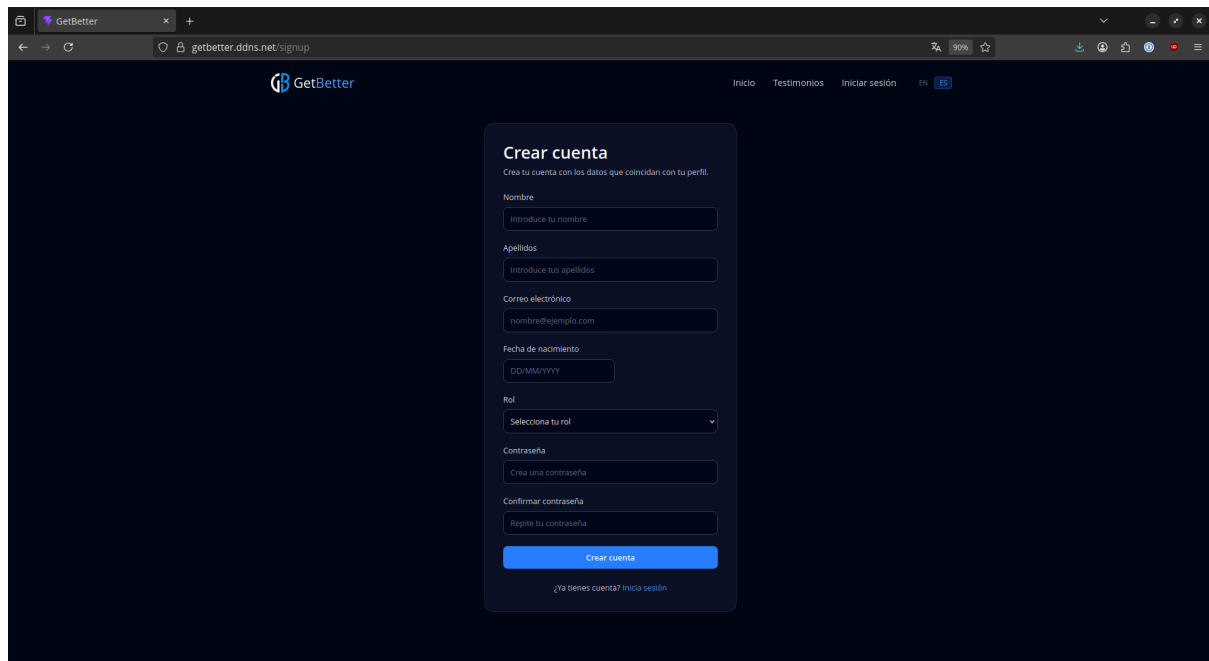


Imagen 5. Captura de pantalla de app en producción (registro de usuario)

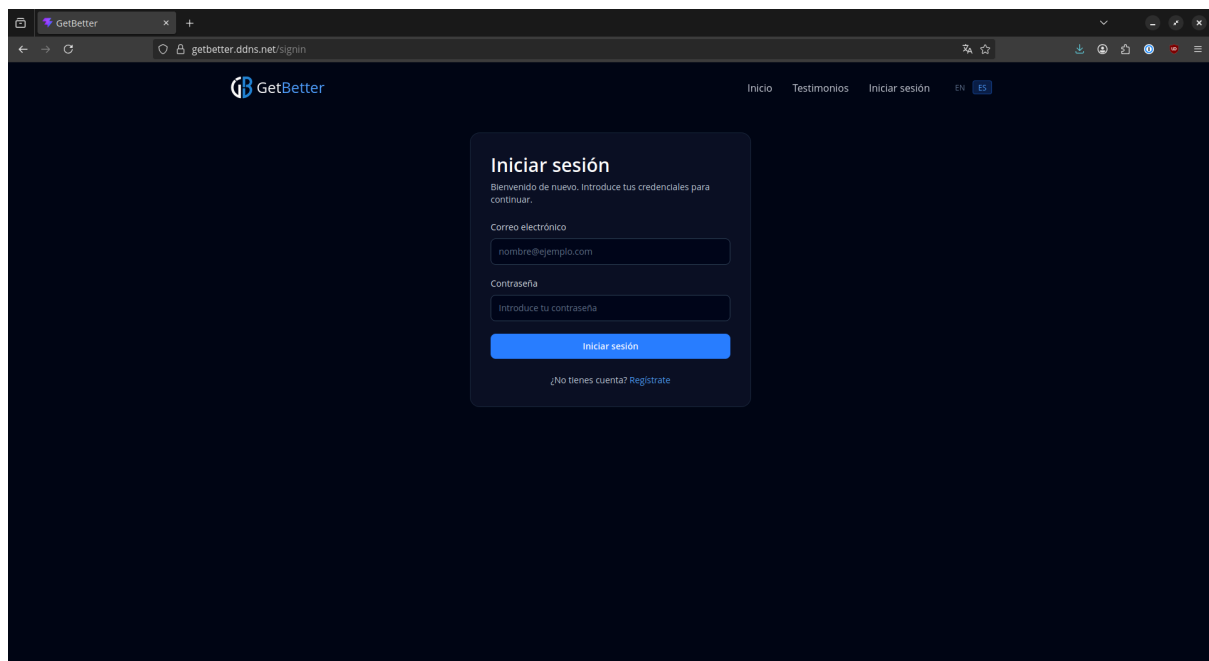


Imagen 6. Captura de pantalla de app en producción (inicio de sesión)

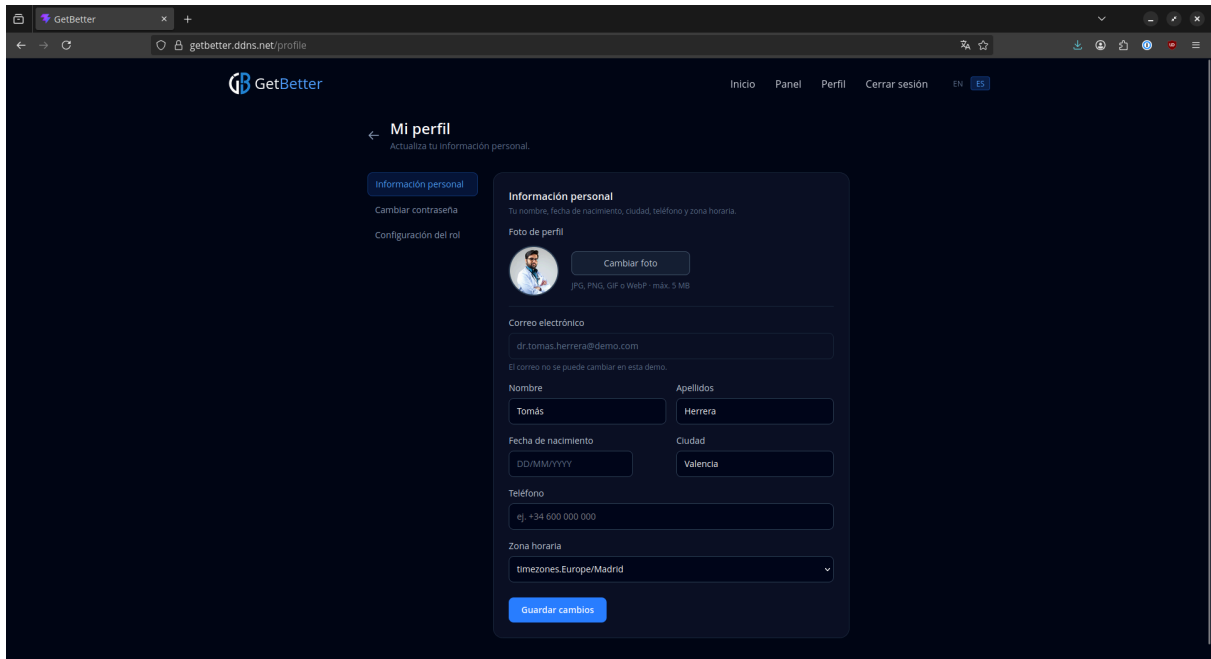


Imagen 7. Captura de pantalla de app en producción (gestión de perfil)

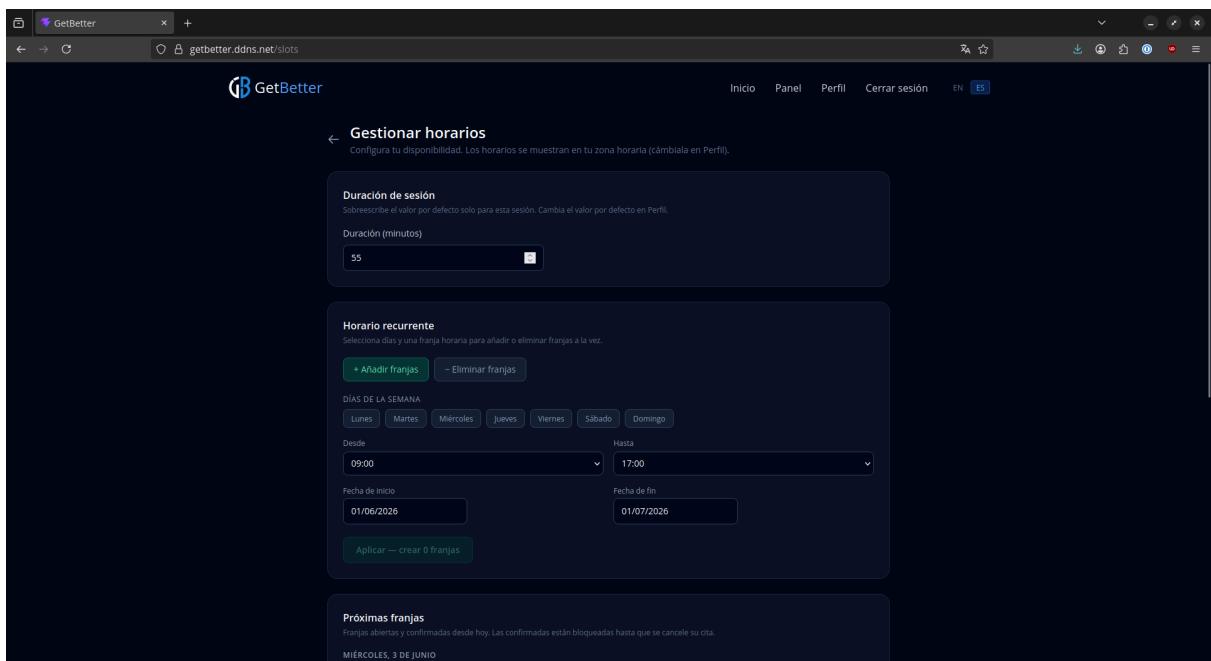


Imagen 8. Captura de pantalla de app en producción (gestión de horarios - solo para psicólogos)

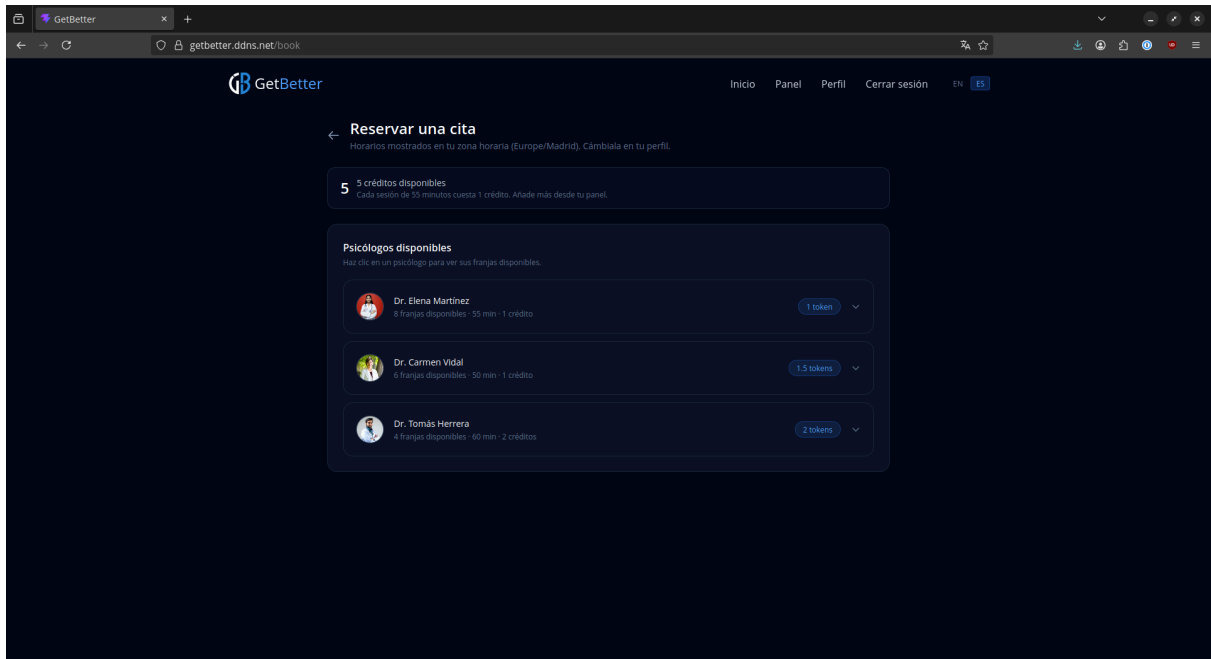


Imagen 9. Captura de pantalla de app en producción (reservas, sólo para pacientes)

Stack de la aplicación web

A nivel general, la pila de aplicaciones es la siguiente:

- Debian como sistema operativo
- Nginx como servidor web
- ReactJS+Tailwind como frontend
- Python/Django + Gunicorn como backend
- Redis
- MariaDB

Frontend: Nginx sirviendo un build de ReactJS

La interfaz de usuario está construida con **ReactJS**, una biblioteca de JavaScript desarrollada por Meta para crear aplicaciones de página única (SPA) basadas en componentes. Para producción, el código fuente de React se compila en un conjunto de archivos estáticos (HTML, CSS y bundles de JavaScript). Estos archivos estáticos son servidos por **Nginx**, un servidor web de alto rendimiento.

Tailwind es un framework de CSS, moderno y fácil de escribir.

Nginx en el nodo de aplicación cumple una doble función: sirve el build estático de React para todas las rutas que no son de API, y actúa como **reverse proxy** para cualquier petición que comience por “/api/”, “/admin”, “/test/” o “/static/”, reenviándola al backend de Django que se ejecuta localmente en el mismo stack de contenedores.

Backend: Django a través de Gunicorn

El backend es una aplicación **Django** (framework web en Python). Django gestiona toda la lógica de negocio, la autenticación de usuarios, la validación de datos, y expone una REST API que el frontend de React consume. En esta arquitectura Django no sirve páginas web directamente; únicamente responde a llamadas de API.

Django se ejecuta dentro de un contenedor Docker a través de **Gunicorn**, un servidor HTTP WSGI (Web Server Gateway Interface) de Python orientado a producción. Gunicorn escucha en el puerto 8000 dentro del contenedor. Nginx redirige a él las peticiones relevantes.

Base de datos: MariaDB

Los datos relacionales persistentes se almacenan en **MariaDB**, una base de datos relacional de código abierto que es un fork de MySQL mantenido por la comunidad. Se ejecuta en una instancia EC2 privada dedicada.

Caché y almacén de sesiones: Redis

Redis es un almacén de datos en memoria de tipo clave-valor que se usa por dos motivos. El primero es almacenar las **sesiones** de usuario de Django: cuando un usuario inicia sesión, Django guarda los datos de sesión en Redis en lugar de en la base de datos, lo que resulta significativamente más rápido para operaciones de lectura intensiva.

El segundo propósito es servir como backend de **caché** de propósito general, reduciendo llamadas repetidas a la base de datos para consultas frecuentes.

Almacenamiento persistente de imágenes: AWS S3

Las imágenes subidas por los usuarios (fotos de perfil) se almacenan en un bucket de **AWS S3** en lugar del sistema de archivos local. Esto es esencial en un entorno multi-nodo: si un usuario sube un archivo al Nodo App 1 y su siguiente petición se enruta al Nodo App 2, el archivo no estaría ahí. S3 proporciona una ubicación de almacenamiento única, compartida y duradera, accesible por todos los nodos de aplicación.

Las librerías de Python que se usan son “boto3” y “django-storages” para el almacenamiento, además de “pillow” para poder procesar imágenes.

Diseño de la base de datos

Django y el ORM

Django utiliza un **ORM (Object-Relational Mapper)**, lo que significa que se definen las tablas de la base de datos como clases de Python llamadas **Models**. Cuando se ejecutan las “migraciones”, Django traduce esas definiciones de clases Python en sentencias SQL “CREATE TABLE” reales y las aplica a la base de datos. No es necesario escribir SQL directamente para crear o modificar tablas.

Esta es una de las características más potentes de Django: aprovecha que un objeto en OOP normalmente es equivalente a una tabla en la base de datos, y además añade control de versiones. Si las tablas necesitan cambiar, las migraciones permiten que el código haga esos cambios automáticamente mediante una migración, que además pueden revertirse.

Los modelos de datos

La primera tabla y más importante es la de Users, que contiene toda la información de usuario que necesita Django. Todos los valores están normalizados y son dependencia exclusiva de la clave primaria, aunque según expandamos la definición de datos de la tabla, es muy probable que haya que reestructurar el modelo para mantener normalización de quinta forma.

Modelo ER ([Link a Excalidraw](#))

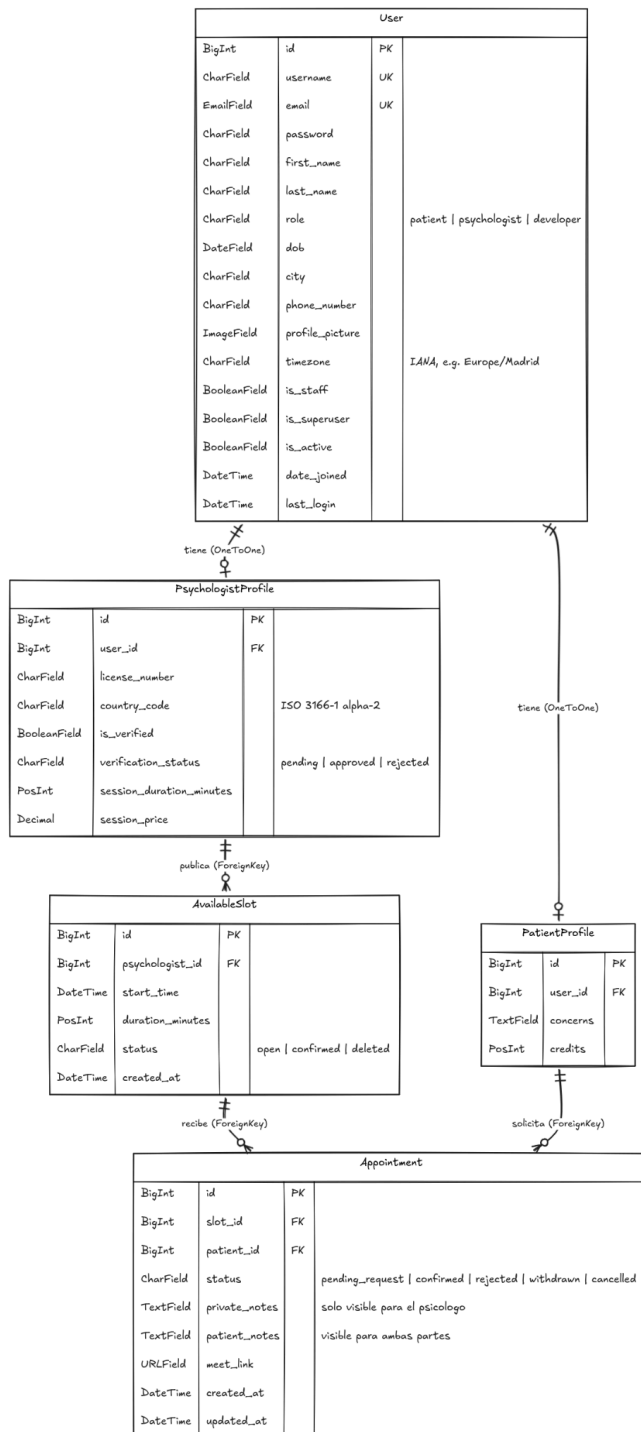


Imagen 10. Modelo ER de GetBetter. Disponible en mayor tamaño en Anexo II

La tabla Users es heredada por otras dos tablas, pacientes y psicólogos. Cada una de esas tablas referencia a su ID en la tabla user y aporta información adicional.

Los psicólogos pueden crear “slots”, huecos donde se pueden citar los pacientes con ellos. La relación es uno-muchos entre psicólogos y slots. En cada slot, pueden pedir cita múltiples pacientes, con lo cual la relación huecos-solicitudes es uno-muchos. Cada paciente puede hacer múltiples solicitudes. No existe relación muchos-muchos entre psicólogos y pacientes, ya queda normalizada en la tabla “appointments” (sesiones).

Diagrama de relaciones

User (1) <-----OneToOne-----> PatientProfile (1)

User (1) <-----OneToOne-----> PsychologistProfile (1)

PsychologistProfile (1) <-----ForeignKey-----> AvailableSlot (many)

AvailableSlot (1) <-----ForeignKey-----> Appointment (many)

PatientProfile (1) <-----ForeignKey-----> Appointment (many)

Ejemplo: flujo completo de una reserva en la base de datos

1. **La Dra. Elena Martínez** se registra. Se crea una fila en `User` con `role=psychologist`. Se crea una fila en `PsychologistProfile` vinculada a ese usuario con `verification_status=pending`.
2. Un administrador verifica su número de colegiado. La fila de `PsychologistProfile` se actualiza: `is_verified=True`, `verification_status=approved`.
3. Elena crea una franja para el próximo martes a las 10:00. Se crea una fila en `AvailableSlot` con `status=open`, vinculada a su `PsychologistProfile`.
4. **Sofía Navarro** (una paciente) solicita esa franja. Se crea una fila en `Appointment` con `status=pending_request`, vinculando el `PatientProfile` de Sofía con el `AvailableSlot` de Elena.
5. Elena confirma la reserva. La fila de `Appointment` se actualiza a `status=confirmed`. La fila de `AvailableSlot` se actualiza a `status=confirmed`.
6. Tras la sesión, Elena escribe sus notas clínicas. La fila de `Appointment` se actualiza con contenido en `private_notes`.

En SQL puro, una consulta para obtener todas las citas confirmadas y futuras de un psicólogo concreto tendría este aspecto:

```
SELECT u.first_name, u.last_name, s.start_time, a.patient_notes
```

```
FROM appointments_appointment a

JOIN appointments_availableslot s ON a.slot_id = s.id

JOIN accounts_patientprofile pp ON a.patient_id = pp.id

JOIN accounts_user u ON pp.user_id = u.id

JOIN accounts_psychologistprofile psy ON s.psychologist_id = psy.id

WHERE psy.user_id = 42

AND a.status = 'confirmed'

AND s.start_time > NOW()

ORDER BY s.start_time ASC;
```

El mismo resultado expresado con el ORM de Django:

```
Appointment.objects.filter(

    slot__psychologist__user=request.user,

    status='confirmed',

    slot__start_time__gt=timezone.now()

).select_related('patient__user', 'slot').order_by('slot__start_time')
```

El panel de administración de Django

Django incluye una interfaz de administración integrada accesible en `/admin/`. Un usuario admin puede acceder a este panel e interactuar con todos los modelos registrados a través de una interfaz gráfica, sin necesidad de escribir SQL ni código.

A través del panel de administración, los administradores del sitio pueden:

- Ver, buscar, filtrar, crear, actualizar y eliminar cualquier registro de la base de datos
- Verificar cuentas de psicólogos cambiando el valor booleano “`is_verified`” y “`verification_status`”
- Inspeccionar el historial y los estados de las citas de cualquier paciente o psicólogo (pero no las notas privadas de las sesiones)
- Gestionar cuentas de usuario, restablecer contraseñas y controlar permisos de staff (pero no puede ver contraseñas en texto plano)
- Supervisar los datos de demostración cargados mediante `python manage.py seed_demo`

El panel de administración es especialmente valioso durante el despliegue inicial para la gestión manual de datos y las pruebas antes de que los flujos de autoservicio estén completamente desarrollados en el frontend.

Utilidades de base de datos

utils/backup_db.sh

Este script de Bash genera un volcado SQL comprimido de la base de datos MariaDB. Funciona en producción y también se hizo una versión local para verificar. Para lanzarlo se lanza “./utils/backup_db.sh cloud”:

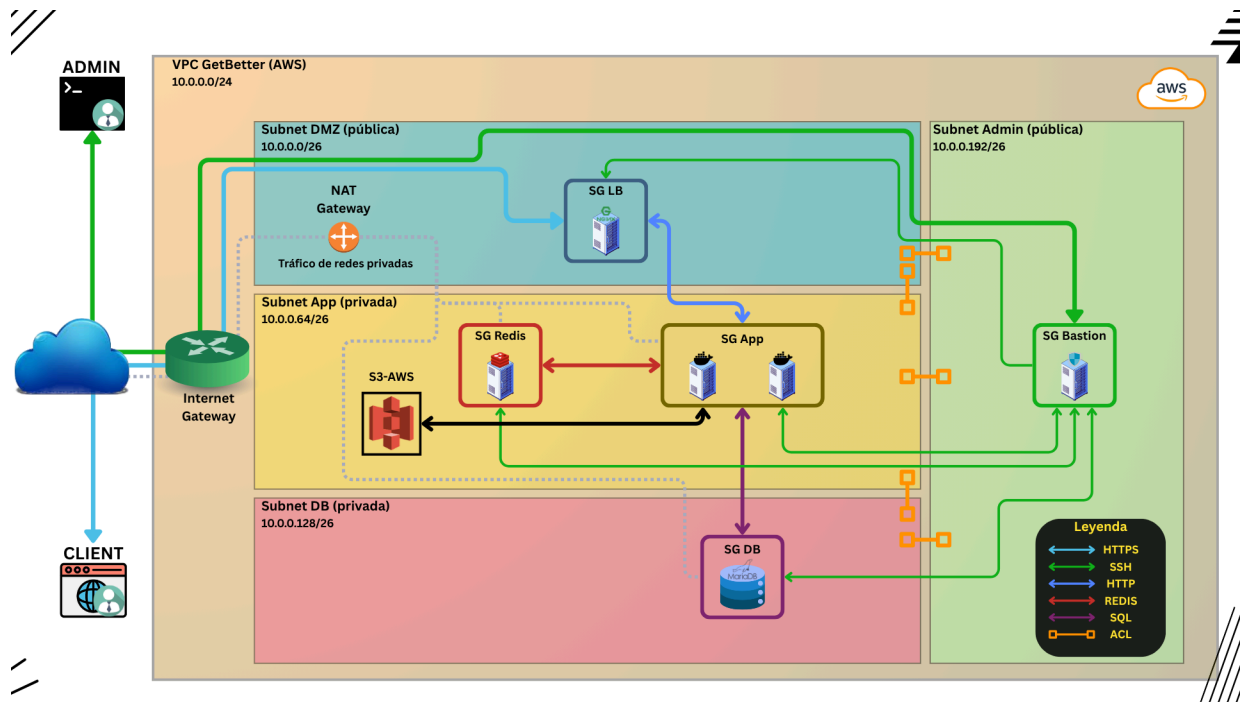
El script se conecta al servidor de base de datos en producción via SSH a través del Bastion host, ejecuta `mariadb-dump` en la máquina remota, transmite la salida de vuelta por SSH y la comprime localmente. El script no requiere contraseña de base de datos en modo cloud porque MariaDB en la instancia EC2 está configurado con autenticación por socket para el usuario root.

Los archivos de backup reciben el nombre `getbetterDB_YYYY-MM-DD_HH-MM-SS_modosql.gz` y están excluidos del control de versiones mediante `.gitignore`.

utils/read_or_unzip_backups.sh

Una pequeña utilidad interactiva que acepta un archivo de backup `.sql.gz` como argumento y ofrece dos opciones: leer el contenido directamente en el terminal (usando `zcat`) sin descomprimir en disco, o descomprimir el archivo a un archivo `.sql` junto al original (usando `gunzip -k` para conservar el original).

Arquitectura AWS



Virtual Private Cloud (VPC)

Toda la infraestructura reside dentro de una única VPC (Virtual Private Cloud) llamada **VPC GetBetter**, con bloque CIDR **10.0.0.0/24**. Una VPC es una sección lógicamente aislada de la red de AWS. Ningún tráfico entra o sale de la VPC a menos que lo permitan explícitamente las reglas de enrutamiento, los Security Groups y las Network ACLs.

Subredes

La VPC se divide en cuatro subredes, cada una sirviendo a una zona de seguridad distinta:

Subnet DMZ (pública) - 10.0.0.0/26

Es la única subred accesible desde la internet pública. Contiene la instancia EC2 del **Load Balancer**, que es el único punto de entrada para todo el tráfico de usuarios, y el **NAT Gateway**, que permite a las instancias en subredes privadas iniciar conexiones salientes (p. ej. para instalar paquetes o ejecutar `git clone`) sin ser accesibles desde internet.

Subnet App (privada) - 10.0.0.64/26

Contiene los nodos de aplicación (App1 y App2), la instancia de Redis y la interfaz con S3. Las instancias aquí no tienen dirección IP pública. Solo reciben tráfico del Load Balancer a través de su Security Group.

Subnet DB (privada) - 10.0.0.128/26

Contiene la instancia de MariaDB. Sin IP pública ni acceso directo a internet. Solo los nodos de aplicación tienen permiso para conectarse en el puerto 3306.

Subnet Admin (pública) - 10.0.0.192/26

Contiene el **Bastion host**. Es una instancia EC2 endurecida utilizada exclusivamente como punto de acceso SSH para la administración. Los administradores se conectan al Bastion desde su propia IP y luego saltan por SSH desde el Bastion a cualquier instancia privada. El Bastion tiene IP pública y solo acepta SSH desde direcciones IP de administradores en lista blanca.

Flujo de tráfico

Viente tráfico de Internet (HTTPS puerto 443). Entre en la Internet Gateway.

Lo recibe el Load Balancer (Subnet DMZ, pública)

- Nginx con terminación SSL mediante Certbot / Let's Encrypt
- Comprueba el `least_conn` (que nodo está menos ocupado) y lo envía al upstream que toque. Reenvía HTTP, no HTTPS.

Llega el tráfico a un Nodo App (Subnet App, privada). El contenedor Nginx devuelve estáticos de ReactJS (SPA). Aún así, seguirá recibiendo del cliente llamadas a `/api/`, que redirige al contenedor de Django.

Aún en Nodo App, el contenedor de Django procesa las peticiones de la API, y se comunica con Redis para leer o escribir sesiones mediante puerto 6379, o lee y escribe datos persistentes en MariaDB puerto 3306, o lee y escribe imágenes en AWS S3.

Escalabilidad

La arquitectura es horizontalmente escalable por diseño. La capa de aplicación es sin estado (stateless): todos los datos de sesión viven en Redis (compartido por ambos nodos), todas las subidas de archivos van a S3, y todos los datos persistentes están en MariaDB. Esto significa que se puede añadir un nuevo nodo de aplicación en cualquier momento:

1. Lanzar una nueva instancia EC2 en la Subnet App
2. Ejecutar `app_setup.sh` en ella
3. Ejecutar `app_deploy.sh` en ella
4. Añadir su IP privada al bloque upstream de Nginx del Load Balancer usando `update_lb.sh`

No se requiere tiempo de inactividad ni migración de datos. El script `setup_or_deploy_EC2_aws.sh` admite un número arbitrario de entradas `APP_IP_N` en el mapa de configuración e iterará sobre todas ellas en operaciones masivas.

La base de datos y Redis pueden escalarse de forma independiente (verticalmente redimensionando la instancia EC2, u horizontalmente migrando a réplicas de lectura o Redis Cluster respectivamente) sin tocar el código de la aplicación.

Security Groups y Network ACLs

Los **Security Groups** actúan como cortafuegos virtuales con estado a nivel de instancia. Cada instancia pertenece a uno o más Security Groups, y cada grupo define qué tráfico de entrada y salida está permitido.

- **SG LB:** acepta HTTPS (443) desde cualquier origen (`0.0.0.0/0`) y SSH (22) solo desde el SG del Bastion.
- **SG App:** acepta HTTP (80) solo desde el SG LB, y SSH (22) solo desde el SG Bastion.
- **SG Redis:** acepta el puerto 6379 solo desde el SG App.
- **SG DB:** acepta el puerto 3306 solo desde el SG App.
- **SG Bastion:** acepta SSH (22) solo desde las IPs de los administradores.

Las **Network ACLs (NACLs)** son reglas de cortafuegos sin estado aplicadas a nivel de subred, que proporcionan una segunda capa de control de acceso independiente de los Security Groups. Las NACLs evalúan las reglas en orden numérico y ofrecen un mecanismo explícito de permitir/denegar para el tráfico que cruza los límites de las subredes, complementando los Security Groups a nivel de instancia.

Este enfoque por capas significa que aunque un Security Group estuviese mal configurado, la NACL seguiría aplicando los límites de subred, y viceversa.

Orquestación y scripts de despliegue

Todos los scripts de despliegue se encuentran en el directorio `deployment/`. Están diseñados para ejecutarse desde la máquina local del administrador, conectándose a las instancias EC2 de forma remota por SSH a través del Bastion host.

Configuración del entorno

Antes de que cualquier script pueda ejecutarse, el administrador debe completar dos archivos clave:

- `deployment/.aws-map.env` - mapea nombres lógicos a las direcciones IP reales y nombres de host de todas las instancias EC2, más la ruta al archivo de clave SSH.
- `app/.app.base.env` y `app/.app.aws.env` - contienen las variables de entorno a nivel de aplicación (credenciales de base de datos, contraseña de Redis, clave secreta de Django, detalles del bucket S3, etc.).

Estos archivos están excluidos del control de versiones por razones de seguridad.

`deployment/setup_or_deploy_EC2_aws.sh`

Es el **script maestro de orquestación**. Es el punto de entrada único para todas las operaciones de configuración y despliegue. Al ejecutarse, presenta un

menú interactivo que permite al administrador elegir sobre qué componente actuar: Bastión, Load Balancer, base de datos, Redis o uno o varios nodos App.

Para cada componente ofrece sub-opciones: bien una configuración completa (para una instancia EC2 nueva) o una actualización específica (p. ej. actualizar solo las IPs del upstream en el Load Balancer). Lee la topología desde `.aws-map.env`, establece conexiones SSH a través del Bastion usando `ProxyJump`, y envía por pipe el sub-script apropiado a la máquina remota para su ejecución. Las variables de entorno sensibles se ensamblan localmente en un archivo temporal, se transfieren via `scp` a través del Bastion y se eliminan al terminar.

deployment/lb/lb_setup.sh

Configura una instancia EC2 Debian nueva como Load Balancer con Nginx. Establece el hostname, instala Nginx y Certbot, genera una configuración de Nginx con un bloque upstream apuntando a todos los nodos app (usando el algoritmo `least_conn`), y provisiona un certificado TLS/SSL gratuito de Let's Encrypt via Certbot, configurando la redirección automática a HTTPS.

deployment/lb/update_lb.sh

Actualiza únicamente el bloque upstream de un Load Balancer ya configurado. Este es el script que se ejecuta al añadir o eliminar nodos de aplicación. Reescribe la sección upstream de la configuración de Nginx in sitio usando `awk`, crea una copia de seguridad con marca de tiempo de la configuración anterior, valida la nueva configuración con `nginx -t`, y recarga Nginx sin tiempo de inactividad usando `systemctl reload nginx`.

deployment/db/db_setup.sh

Configura una instancia EC2 Debian nueva como servidor de base de datos MariaDB. Instala MariaDB, crea la base de datos de la aplicación y crea un usuario de base de datos para cada IP de nodo de aplicación (ya que el sistema de usuarios de MariaDB incluye la IP de origen en la credencial), concediendo a ese usuario todos los privilegios sobre la base de datos de la aplicación. También configura MariaDB para escuchar en todas las interfaces (0.0.0.0) para que los nodos app en la subred privada puedan conectarse.

deployment/redis/redis_setup.sh

Configura una instancia EC2 Debian nueva como servidor Redis. Instala Redis, lo configura para escuchar en todas las interfaces, activa la persistencia AOF (Append-Only File) para que los datos de sesión sobrevivan a un reinicio, establece una contraseña y reinicia el servicio.

deployment/app/app_setup.sh

Prepara una instancia EC2 Debian nueva para alojar la aplicación. Establece el hostname, actualiza el sistema e instala Docker y Docker Compose usando el procedimiento oficial de instalación de Docker para Debian. También instala Git. Este script solo instala dependencias; no clona el repositorio ni inicia ningún contenedor.

deployment/app/app_deploy.sh

Despliega la aplicación en un nodo app previamente configurado. Clona el repositorio si aún no está presente (o ejecuta `git pull` para actualizarlo), elimina los directorios de despliegue e infraestructura local que no son necesarios en producción, recupera el archivo `.env.runtime` de `/tmp/` (donde `setup_or_deploy_EC2_aws.sh` lo dejó via `scp`), detiene los contenedores en

ejecución, elimina imágenes Docker no usadas e inicia el stack de la aplicación con `docker compose up -d --build`.

`deployment/bastion/bastion_setup.sh`

Establece el hostname en el Bastion host y realiza pasos básicos de endurecimiento. El papel principal del Bastion es ser una pasarela SSH controlada; este script asegura que esté correctamente identificado dentro de la red.

`deployment/jump_to_EC2_aws.sh`

Una utilidad de conveniencia para administradores. Presenta un menú interactivo con todas las instancias EC2 de la topología y abre una sesión SSH interactiva directa a la máquina elegida, enrutando automáticamente a través del Bastion mediante `ProxyJump` y cargando la clave SSH en `ssh-agent`.

Infraestructura local (Windows Server)

La infraestructura empresarial interna refleja el despliegue tecnológico típico de cualquier empresa que tiene que trabajar con ordenadores y usuarios no dedicados a la tecnología. Esta empresa será simulada y virtualizada en un equipo portátil de 16GB RAM, en una red interna de VirtualBox.

Windows Server - Active Directory

Se implementa un dominio corporativo con:

- Una OU por cada equipo o unidad organizativa.
- Creación de:
 - Usuarios
 - Grupos
 - Asociación a equipos físicos
- Control estricto de permisos para proteger datos sensibles.

Estructura organizativa - Grupos globales

- Directivos
- Admins
- JefeArea_Global
- Trabajadores_Global
- Becarios_Global

Estructura organizativa - Grupos dentro de unidades organizativas anidadas:

OU - Producto Tecnológico

Grupos:

- Desarrollo Web
- Becarios_DWEB

OU - Desarrollo Personal

Grupos:

- Psicólogos
- Becarios_PSICO

OU - Márketing y RRSS

Grupos:

- Márketing
- Becarios_MARK

OU - Contable y Finanzas

- Contable
- Becarios_FINA

OU - Gestores, administrativos y RRHH

- Administrativo
- Becarios_GEST

Windows Server - Otros servicios integrados

- Servicio DHCP
- Servicio DNS

Servicios en Windows Server 2022

Active Directory Domain Services (AD DS)

El Windows Server actúa como Controlador de Dominio para la red interna de la clínica. Mantiene el directorio autoritativo de todos los usuarios corporativos (p. ej. personal administrativo, psicólogos empleados directamente por la clínica). El dominio proporciona un sistema centralizado de gestión de identidades.

DHCP (Dynamic Host Configuration Protocol)

El rol de servidor DHCP está configurado en Windows Server para asignar automáticamente direcciones IP, máscaras de subred, puertas de enlace predeterminadas y direcciones de servidores DNS a las máquinas cliente que se unen a la red del dominio. Esto elimina la necesidad de configurar la red manualmente en cada puesto de trabajo.

DNS (Domain Name System)

Windows Server aloja la zona DNS interna para el dominio de la clínica. Resuelve nombres de host internos (p. ej. `fileserver.clinic.local`) para las máquinas unidas al dominio, y reenvía las consultas DNS externas a un servidor resolovedor superior. La infraestructura de Active Directory depende del DNS para funcionar.

FTP (File Transfer Protocol)

Se configura un servicio FTP para proporcionar un punto de transferencia de archivos controlado para la red interna de la clínica. Una mejor alternativa sería NTFS compartido, pero es obligatorio desplegar FTP en este proyecto.

Configuración de Active Directory

Group Policy Objects (GPOs)

- Una GPO de **configuración de escritorio corporativo** que establece fondos de pantalla, restringe el acceso a la configuración del sistema y aplica un diseño estándar del menú de inicio adecuado para un entorno clínico.
 - Una GPO de **restricción de ejecución de PowerShell** que impide a los usuarios ejecutar scripts de PowerShell, reduciendo la superficie de ataque para amenazas basadas en scripting en las máquinas de los usuarios finales.
-

Seguridad

Descripción general

GetBetter maneja datos personales y clínicos sensibles. Hay dos marcos regulatorios principales que obligan a tener medidas de seguridad técnicas y organizativas específicas: el **Reglamento General de Protección de Datos (RGPD)** y el **Esquema Nacional de Seguridad (ENS)**, el marco de seguridad nacional de España para sistemas de información. Por ahora, no se ha profundizado en los requisitos específicos, pero sí se han valorado riesgos.

Análisis de riesgos

Los activos principales que requieren protección son:

- **Identidades y datos personales de los pacientes** (nombre, fecha de nacimiento, datos de contacto)
- **Notas clínicas de sesión** (especialmente las privadas del psicólogo sobre el paciente, que son estrictamente confidenciales)
- **Credenciales de autenticación** (contraseñas, tokens de sesión)
- **Datos profesionales y de colegiado del psicólogo**

Los principales vectores de amenaza son:

- Acceso no autorizado a la base de datos desde internet
- Interceptación de datos en tránsito
- Compromiso de credenciales por fuerza bruta o phishing
- Acceso interno más allá de los límites del mínimo privilegio
- Acceso físico al hardware del servidor

Seguridad de red por capas

Zonas de disponibilidad y redundancia: la aplicación se ejecuta en dos nodos App independientes. Un fallo en un nodo hace que el Load Balancer enrute automáticamente todo el tráfico al nodo superviviente. Desplegar los nodos en zonas de disponibilidad de AWS separadas proporciona aislamiento físico ante fallos a nivel de centro de datos, asegurando que la plataforma permanezca operativa incluso durante interrupciones parciales de la infraestructura.

Subredes privadas: las instancias de base de datos y Redis no tienen direcciones IP públicas y son inalcanzables desde internet. Solo pueden recibir conexiones de los nodos de aplicación dentro de la misma VPC.

Security Groups: cada instancia EC2 tiene un Security Group dedicado que solo permite el tráfico entrante mínimo necesario. La base de datos acepta conexiones solo desde las IPs de los nodos de aplicación. Redis acepta conexiones solo desde los nodos de aplicación. El Load Balancer acepta HTTPS desde cualquier origen pero SSH solo desde el Bastion.

Network ACLs: una segunda capa de control de acceso sin estado aplicada en el límite de la subred, proporcionando defensa en profundidad independiente de la configuración de los Security Groups.

Bastion host: ninguna instancia EC2 en una subred privada es accesible directamente desde internet. Todo el acceso SSH administrativo fluye a través del Bastion, que es el único punto de administración expuesto públicamente. Esto concentra la superficie de ataque para el acceso administrativo en un único punto monitorizado.

TLS/SSL en tránsito: todo el tráfico entre los usuarios finales y el Load Balancer está cifrado mediante HTTPS, con un certificado TLS provisionado por Let's Encrypt. El tráfico interno entre el Load Balancer y los nodos app viaja sobre la red privada de la VPC, que está aislada de la internet pública.

Protección de datos

Contraseñas con hash: el framework de autenticación de Django nunca almacena contraseñas en texto plano. Las contraseñas se hashean usando PBKDF2 con SHA-256 por defecto, con múltiples rondas de iteración, antes de escribirse en la base de datos. Incluso una filtración completa de la base de datos no expondría las contraseñas en bruto.

Control de acceso en la capa de aplicación: la REST API de Django aplica autenticación y autorización en cada endpoint. Los pacientes solo pueden ver sus propias citas. Los psicólogos solo pueden ver franjas y citas vinculadas a su propio perfil. Las notas clínicas privadas nunca se incluyen en las respuestas de la API enviadas a los pacientes.

Relevancia del RGPD: la plataforma recoge y procesa datos personales (identidad, fecha de nacimiento, datos de sesión relacionados con la salud). Bajo el RGPD, esto constituye datos de categoría especial, que requieren consentimiento explícito, minimización de datos, derecho de supresión y salvaguardas técnicas adecuadas. La arquitectura da soporte a estos requisitos mediante acceso controlado, tránsito cifrado y la capacidad de eliminar registros de usuario.

Relevancia del ENS: como plataforma potencialmente utilizada por profesionales afines a la salud pública en España, el cumplimiento del Esquema Nacional de Seguridad es relevante. El ENS exige controles en torno a la gestión de accesos, el registro de auditoría, la respuesta a incidentes y la planificación de la continuidad, muchos de los cuales están abordados por la arquitectura descrita en este documento y se desarrollarán con más detalle en iteraciones futuras.

Desarrollo de software

Django (Python)



Django es un framework web de alto nivel para Python que sigue el patrón **MTV (Model-Template-View)**. En este proyecto la capa Template no se usa (React gestiona todo el renderizado de la interfaz), pero las capas Model y View son centrales.

Los **Models** definen el esquema de la base de datos como clases Python. Las **Views** (más precisamente llamadas ViewSets o API Views en el contexto de Django REST Framework) gestionan las peticiones HTTP entrantes, aplican la lógica de negocio, interactúan con los modelos y devuelven respuestas JSON al

frontend de React. Es decir, a pesar de su nombre, es importante no confundir una ‘Vista’ de Django (lógica) con una ‘Vista’ de Modelo-Vista-Controlador (que realmente hace UI/UX)

El archivo de configuración de Django **settings.py** se ha configurado pasando variables de entorno, en lugar de “hardcodear” los valores en el archivo.

Esto es muy importante porque así tenemos las variables de entorno como única fuente de verdad, y otros archivos de configuración también “beben” de esas variables. De esta forma, es mucho más fácil hacer cambios a futuro, ya que solo habría que cambiar las variables de entorno, y tanto django como nginx responderían a nuestras necesidades.

También nos permite mantener un despliegue local medianamente funcional – salvo por el cubo S3, que no se ha replicado en volumen docker. El mismo código puede ejecutarse localmente contra SQLite sin Redis ni S3, y en producción contra MariaDB, Redis y S3, simplemente cambiando el archivo de entorno.

Django aplica cuatro validadores de contraseña por defecto (similitud con el nombre de usuario, longitud mínima, lista de contraseñas comunes, comprobación de solo números), garantizando una calidad mínima de contraseña a nivel de aplicación.

ReactJS



React es una biblioteca de JavaScript para construir interfaces de usuario basadas en componentes. En lugar de que el servidor renderice HTML en cada petición (como harían los templates de Django), React se ejecuta completamente en el navegador. En la primera carga, Nginx entrega el bundle de JavaScript compilado. React gestiona entonces toda la navegación, el estado y el renderizado en el lado del cliente, comunicándose con el backend de Django exclusivamente a través de llamadas a la API.

Esta arquitectura produce una experiencia de usuario rápida y fluida porque las transiciones de página no requieren un viaje completo de ida y vuelta al servidor.

Pero, lo más importante de todo, es que esto reduce la carga de trabajo sobre nuestro servidor, que pasa de tener que renderizar páginas completas a solamente responder a llamadas API. Es el cliente, en su navegador, quien computa la UI/UX.

Como se manda toda la interfaz web ya “hecha”, el tiempo de carga puede ser mayor. Se ha aliviado este problema creando una pantalla de carga con poquito código en index.html, lo que consigue que tan solo con unos cientos de bytes, el usuario ya pueda empezar a ver que la página carga, en lugar de tener que esperar al medio megabyte que pesa ReactJS en estos momentos, además de las imágenes que acompañan a la página de inicio.

Funciones de React usadas en este proyecto:

- Hooks `useState` y `useEffect` para gestionar el estado a nivel de componente y los efectos secundarios
- React Router para la navegación en el lado del cliente entre páginas (casi toda la navbar login, registro, dashboard, perfil, citas)
- Context API (`AuthContext`) para compartir el estado de autenticación (el objeto del usuario autenticado) por todo el árbol de componentes sin prop drilling. Se llama, en esencia, en cada página donde se necesita saber si el usuario está logueado o no.
- Funciones de servicio personalizadas que envuelven llamadas “fetch” a la API de Django

i18n (Internacionalización)



Conocí i18n en previas experiencias en open source, colaborando como colaborador. Decidí implementarlo en este proyecto. i18next es la biblioteca de traducción programática más utilizada. Todos los textos visibles para el usuario están externalizados en archivos JSON. Si se necesita dar la bienvenida al usuario, se mira el respectivo JSON, por ejemplo, el subtítulo de la página principal está en dos archivos:

- `app/react/getbetter/src/i18n/locales/es/common.json`
- `app/react/getbetter/src/i18n/locales/en/common.json`

Dentro, están los diccionario con las traducciones, de manera que en el código solo se invoca la llamada a la cadena que se quiere mostrar, no la cadena en sí.

Es muy escalable. Añadir un nuevo idioma solo requiere una nueva carpeta de idioma y archivos JSON, sin ningún cambio en el código de los componentes.

Esto es importante para una plataforma que atiende tanto a profesionales clínicos hispanohablantes como a usuarios internacionales.

Validación en el lado del cliente

La validación de entradas ocurre en dos niveles: el frontend (React) y el backend (Django). La validación en el lado del cliente proporciona retroalimentación inmediata al usuario antes de que se realice ninguna petición de red.

La lógica de validación en `src/utils/validate.js` define:

- **Expresiones regulares** para nombres (letras, guiones, espacios, apóstrofes, incluyendo caracteres españoles acentuados), formato de email y números de teléfono.
- **Filtros de teclas en vivo** (`filterName`, `filterEmail`, `filterPhone`) que eliminan silenciosamente los caracteres no válidos mientras el usuario escribe, impidiendo que se introduzca entrada inválida en absoluto.
- **Validación de edad** (`isOldEnough`) que calcula si una fecha de nacimiento corresponde a alguien con 16 años o más.
- **`validateSignUpValues`** que ejecuta todas las comprobaciones de campos y devuelve un objeto de errores con clave por nombre de campo. Si el objeto no está vacío, el formulario no se envía.

Los mensajes de validación se pasan a través de `t()` para que también estén localizados.

La validación del backend se realiza de forma independiente en los serializadores y vistas de Django, garantizando la seguridad incluso si se saltara el frontend.

GIF de precarga

Mientras el bundle de JavaScript de React se está analizando y la aplicación se está inicializando, el archivo HTML inicial (servido por Nginx) muestra un indicador de carga. Esto evita que el usuario vea una página en blanco durante el breve período de inicialización. Una vez que React monta la aplicación y está lista, el indicador de carga es reemplazado por la interfaz real.

Contraseñas con hash en el registro

Cuando un usuario envía el formulario de registro, la contraseña en bruto se envía por HTTPS al endpoint de la API de Django. El método `create_user()` de Django (o el método `create()` del serializador) llama a `set_password()`, que hashea la contraseña usando el hasher configurado de Django (PBKDF2-SHA256 por defecto) antes de escribirla en la base de datos. La contraseña en bruto nunca se registra en ningún log ni se persiste en ningún lugar.

Scripting en Bash

Toda la automatización de infraestructura está escrita en **Bash**. Los scripts siguen prácticas defensivas: `set -euo pipefail` al comienzo de cada script garantiza que el script se detiene inmediatamente ante cualquier error (`-e`), trata las variables no definidas como errores (`-u`) y propaga correctamente los fallos de pipe (`-o pipefail`). Las variables de entorno se cargan usando `source` con eliminación de comentarios en línea para evitar problemas comunes de análisis. El reenvío del agente SSH permite la autenticación por clave encadenando desde el Bastion a las instancias privadas sin exponer los archivos de clave privada en las máquinas remotas.

Posibles mejoras futuras

Seguridad y cumplimiento normativo

- **Pruebas de penetración:** encargar una prueba de penetración estructurada de la aplicación e infraestructura desplegadas para identificar vulnerabilidades antes de un despliegue más amplio. Abordar los hallazgos y repetir las pruebas.
- **Auditoría de cumplimiento del RGPD:** formalizar un registro de actividades de tratamiento, implementar flujos de consentimiento explícito para la recogida de datos, añadir un mecanismo de derecho de supresión en el panel de administración y potencialmente en el propio perfil del usuario, y elaborar una política de privacidad.
- **Cumplimiento del ENS:** realizar un análisis formal de brechas respecto a los controles de referencia del Esquema Nacional de Seguridad. Implementar registro de auditoría obligatorio de todos los accesos a datos clínicos, establecer un procedimiento de respuesta a incidentes y documentar un plan de continuidad de negocio.
- **WAF y protección contra DDoS:** desplegar un Web Application Firewall (AWS WAF) frente al Load Balancer para detectar y bloquear patrones de ataque comunes (inyección SQL, XSS, inundaciones de peticiones).

Integración de pagos (Stripe)

Integrar la API de pagos de **Stripe** para permitir a los pacientes comprar créditos de sesión y procesar pagos al confirmar una reserva. El campo `credits` en `PatientProfile` y el campo `session_price` en `PsychologistProfile` ya están presentes en el modelo de datos, proporcionando la base para esta funcionalidad. Se añadiría un endpoint de webhook de Stripe a Django para gestionar la confirmación asíncrona de pagos.

Integración de videollamadas (Google Meet API)

El modelo `Appointment` ya tiene un campo `meet_link`. Integrar la **Google Meet API** (o un proveedor similar) permitiría a Django generar automáticamente un enlace de videollamada único cuando se confirma una cita y rellenar este campo, eliminando la necesidad de que el psicólogo cree y comparta manualmente un enlace.

OAuth (inicio de sesión social)

Añadir autenticación basada en OAuth (Google, Microsoft) usando `django-allauth` o `social-django`. Esto permitiría a los pacientes crear cuentas e iniciar sesión usando sus cuentas existentes de Google o Microsoft, reduciendo la fricción en el registro. Es especialmente útil si el OAuth de Microsoft se configura para permitir a los empleados de la clínica autenticarse con sus cuentas corporativas de Microsoft existentes.

Balanceo de carga nativo de AWS (ALB)

Reemplazar la instancia EC2 con el Load Balancer de Nginx configurado manualmente por un **AWS Application Load Balancer (ALB)**. ALB es un servicio completamente gestionado que proporciona comprobaciones de estado automáticas, balanceo de carga entre zonas de disponibilidad, terminación SSL e integración nativa con servicios de AWS como WAF, Certificate Manager y Auto Scaling. Esto eliminaría la necesidad de gestionar una instancia EC2 para el balanceo de carga y aumentaría la fiabilidad.

Orquestación mediante GitHub Actions o Ansible

Incorporar herramientas de orquestación para reducir mantenimiento manual.

Gestión de DNS (Route 53)

Reemplazar la resolución de dominio basada en DDNS actual (getbetter.ddns.net) por **AWS Route 53**, el servicio DNS gestionado de Amazon. Route 53 proporciona resolución DNS fiable y de baja latencia, enrutamiento basado en comprobaciones de estado e integración nativa con ALB y otros servicios de AWS. Se registraría un dominio propio y se gestionaría a través de Route 53.

Aplicación de monitorización de pacientes

Desarrollar una aplicación web o móvil complementaria que permita al psicólogo asignado de un paciente monitorizar pasivamente indicadores conductuales seleccionados reportados por el paciente (similar en concepto al control parental, pero basado en el consentimiento y con orientación terapéutica). Esta aplicación alimentaría de datos a la plataforma principal de GetBetter, dando a los psicólogos una visión longitudinal más rica de los patrones de comportamiento del paciente entre sesiones. Esta es la funcionalidad más directamente alineada con el enfoque de especialización de la plataforma en adicciones conductuales, donde los datos de comportamiento entre sesiones tienen relevancia clínica significativa.

Conclusión general

El proyecto integra un desarrollo full stack con infraestructura en AWS con alta disponibilidad. Se simula un dominio de empresa gracias a VirtualBox y Windows Server+Clientes.

En la aplicación web, ReactJS y Tailwind se encargan de un frontend bonito y llamativo, además de servir como framework para poder incluir a otros desarrolladores en el futuro. El backend es gestionado por Django gracias a Gunicorn, que permiten operaciones CRUD en la base de datos MariaDB, incluso a través de interfaz gráfica.

Por último, se utiliza Docker para reproducibilidad y escalabilidad, scripts Bash para mantenimiento y actualizaciones, una EC2 Redis para mantener vivas las sesiones en memoria caché, un bucket AWS S3 para almacenamiento único de imágenes de perfil, y una EC2 Bastión para gestión SSH segura.

Es por ello un proyecto completo, escalable y que pretende poner la tecnología al servicio de profesionales humanos, para que ayuden a personas a ser más sanas y tener más control en su día a día.

Bibliografía

- <https://mgyf.org/las-adicciones-comportamentales-espana/>
- [Securely Deploy a Django App With Gunicorn, Nginx, & HTTPS – Real Python](#)
-  How to connect Django to MySQL in a Docker Container
- [MariaDB in brief](#)
- [DJANGO Introduction](#)
-  Build and Deploy a Fully Responsive Modern Website using ReactJS...
-  Image File Upload to User Profile Model | Django (3.0) Crash Course...
-  Image File Upload to User Profile Model | Django (3.0) Crash Course...
- [Redis docs](#)
- <https://docs.aws.amazon.com/boto3/latest/reference/services/s3.html>

Anexo I: Requisitos mínimos y cómo se cumplen

En este anexo se detallan los requisitos mínimos de cada asignatura de implementación obligatoria en el proyecto y cómo están cubiertas en mi proyecto, de forma clara y resumida.

Administración de sistemas operativos

1. Uno o varios scripts para la administración de servidor Linux
 - a. Dockerfile que ejecuta varios comandos Bash para montar el contenedor Django
 - b. Múltiples scripts de Bash para configuración inicial y despliegue reiterado de instancias AWS
 - c. Script BASH de migraciones de la BBDD mediante llamada a [manage.py](#), con output a la CLI para supervisar el sistema
2. Dominio de empresa en Windows Server
 - a. Detalladas las OUs, grupos, usuarios y GPOs
 - b. Detallados los servicios implementados
3. Parte Linux opcional. Instalación de servidor LDAP e infraestructura del mismo
 - a. No se implementa OpenLDAP, dado que ya se dispone de gestión del dominio mediante Active Directory en Windows Server. Muy fuera de ámbito del proyecto final.
4. Parte Windows opcional - Desarrollo de script Powershell
 - a. Script powershell para migrar la base de datos

Python

1. Aplicación en Python

- a. La aplicación web desplegada se despliega en su totalidad utilizando python en el backend. Todos el código de control está escrito en python, expandiendo sobre las clases y métodos aportados por el framework Django.

2. Gestión de incidencias mediante entorno gráfico

- a. Cualquier usuario puede reportar un error clicando el botón en la esquina inferior derecha.
- b. Las incidencias reportadas se resuelven en getbetter.ddns.net/bugs, todo funciona mediante la autenticación y bases de datos ya existentes

Seguridad y alta disponibilidad

1. Definición e implementación de políticas de seguridad

- a. Se definen ACLs y grupos de seguridad en AWS, se hashem las contraseñas en el registro de la app. Se establecen medidas básicas de seguridad mediante GPO en infraestructura local.

2. Análisis de vulnerabilidades

- a. Se realiza un primer análisis de vulnerabilidades, definiendo riesgos e impactos
- b. Se menciona la posibilidad de contratar una auditoría externa (pentesting) para detectar vulnerabilidades en nuestra red interna y en nuestra aplicación web

3. Aplicación de reglas ip tables

- a. El router de la red interna deniega todo el tráfico por defecto, permitiendo después tráfico al exterior de la red, y entre equipos de la red interna y el servidor.
- b. Se aplican reglas de firewall mediante políticas de seguridad y compartimentalización de redes en AWS

Diseño, gestión y administración de la base de datos

1. Diseño lógico
 - a. Se diseña un modelo ER para el esquema de la base de datos.
 - b. Se verifica la normalización del modelo
 - c. Se esquematiza visualmente y se presenta
2. Diseño físico
 - a. Se implementa el modelo ER mediante las clases “Modelo” en Django (Models, dentro de esquema Modelo-Vista-Controlador)
 - b. Se procede a la migración de los objetos-(de la clase “modelos”) de Django a objetos (tablas) dentro de una BBDD MariaDB alojada en un servidor distinto (Debian) dentro de la red.
3. Reglas específicas
 - a. Se corta el acceso a la BBDD desde el exterior mediante reglas de enrutamiento en AWS, salvo que sea desde las IPs asociadas a las aplicaciones Django. Sólo puerto 3306.
 - b. Se duplica la misma configuración dentro de MariaDB. Sólo puerto 3306 desde las IPs relevantes.
 - c. Se permite acceso SSH desde la IP bastión de AWS.
 - d. Se loguean intentos fallidos de conexión
 - e. Se permiten actualizaciones mediante puerta de enlace NAT.

Implementación de aplicaciones web

1. Infraestructura en general del despliegue en la web
 - a. Se despliega en AWS un balanceador, al menos dos servidores de backend, un servidor de memoria caché, y un servidor de base de datos. Se dividen en redes separadas y se asegura que la base de datos no está expuesta a la red pública. Se tiene DMZ.
2. Servicios de la intranet simulada (Vagrant/VirtualBox)
 - a. Los servidores DHCP, DNS o FTP, entre otros servicios básicos internos, se virtualizan mediante VirtualBox en una red interna.
3. Máquina Debian sin entorno gráfico
 - a. Todas las máquinas de AWS son máquinas Debian sin entorno gráfico
4. Mejora 1 - Firewall
 - a. Ip tables en el router, ACLs en AWS. Firewall de AWS ya integrado
5. Mejora 2 - DMZ
 - a. Tan sólo el balanceador está expuesto al tráfico del internet, y sólo en los puertos 80 y 443
6. Mejora 3 - Balanceo en la base de datos
 - a. No implementado
7. Mejora 4 - Sistema de archivos compartidos para el backend (NFS o similar)
 - a. AWS Storage para permanencia de imágenes a pesar de balanceo. Github para todas las instancias actualizadas al mismo código fuente.

Servicios en red

1. Servicio DHCP

- a. Asignación automática de IPs en la red interna
 - i. Las IPs 192.168.1.51-192.168.1.200 están libres para asignación dinámica
- b. Reservas para servidores estáticos y equipos críticos
 - i. 192.168.1.1 - Router Debian - Puerta de enlace
 - ii. 192.168.1.2 - Windows Server - DHCP, DNS y AD
 - iii. 192.168.1.3 - Servidor FTP
- c. Opcional - Definición de diferentes rangos de LAN
 - i. 192.168.1.11 - 192.168.1.30 - IPs para gestión remota switches
 - ii. 192.168.1.31 - 192.168.1.50 - Impresoras

2. Servicio DNS

- a. Implementación de dominio interno
 - i. Playbetter.gg
- b. Resoluciones directas e inversas de los nombres de equipo
 - i. Se establecen las siguientes:
 - 1. mainServer.playbetter.gg - 192.168.1.2
 - 2. ftpServer.playbetter.gg - 192.168.1.3
- c. Integración con el servidor web y el servidor FTP
 - i. Se resuelven los dominios de dichos servidores

3. Servicio FTP

- a. Creación de un servidor FTP interno para el intercambio de archivos entre departamentos
 - i. Carpeta 'ANUNCIOS PÚBLICOS' visible a todos los departamentos
- b. Implementación de usuarios con diferentes permisos y directorios personales o compartidos
 - i. Una carpeta por cada departamento, automatización de login FTP mediante AD
- c. Configuración de acceso FTP seguro

4. Servicio HTTP

- a. Despliegue de una página web corporativa en un servidor
 - i. Se despliega página web interna que muestra la carga del servidor principal en un dashboard
 - ii. Se despliega la aplicación web principal en AWS
- b. Acceso interno mediante el dominio gestionado por el DNS
 - i. El DNS redirecciona a la ip pública de la aplicación web, dando redundancia al servicio de registro DNS externo (<https://my.noip.com>)

Anexo II: Imágenes ampliadas

